

Phase 1 - Face-Tracking Webcam - Implementation Plan

100% PC-side prototype (Python), driving the ElectronBot via the C-ABI DLL from the AHK package. No C++ compilation, no firmware changes.

1. Scope and principle

- **Real mechanical axis:** only **horizontal** (waist = joint **j6**, $\pm 90^\circ$). The camera is mounted on the torso; rotating the waist rotates the camera.
- **Vertical axis:** **not** mechanical (no joint tilts the camera) → handled via **digital crop** on the output.
- **Output:** re-centered feed exposed as a **virtual webcam** (OBS Virtual Camera) for Zoom/Teams/OBS.
- **Display:** shows static eyes (each `Sync` re-sends the same image — see C1 in the roadmap).

2. Dependencies

```
pip install opencv-python numpy pyvirtualcam
```

- **OBS Studio** installed (provides the "OBS Virtual Camera" backend that `pyvirtualcam` uses on Windows).
- **Robot DLLs** (from the AHK package, which export `AHK_*`): `3.Software/_Tools/AHK-ExpansionPack/ElectronBotSDK/` → `ElectronBotSDK-LowLevel.dll`, `USBInterface.dll`, `opencv_world455.dll`, `libusb0.dll`.
- **Prerequisite (Phase 0):** driver via Zadig/BotDriver + calibrated servos (ServoToolKit).

⚠ Are the robot DLLs **x86 (32-bit)**? Verify the architecture and use a **Python of the same bitness** so `ctypes` matches. (Check with `dumpbin /headers` or by attempting `LoadLibrary .`)

3. File structure

```

webcam_follow/
├ main.py          # main loop
├ robot.py         # ctypes binding -> AHK_* (hardware layer)
├ tracker.py       # capture + face detection/tracking
├ controller.py    # control loop (error -> yaw j6)
├ vcam.py          # virtual webcam output + vertical crop
├ config.py        # parameters (gains, limits, paths, VID/PID)
├ assets/eyes.jpg  # 240x240 eyes image for the display
└ dll/             # copy of the robot DLLs (same folder)

```

4. robot.py — ctypes binding

```

import ctypes, os

class Robot:
    def __init__(self, dll_dir, dll_name="ElectronBotSDK-LowLevel.dll"):
        os.add_dll_directory(dll_dir)          # finds USBInterface/opencv/libusb
        self.lib = ctypes.CDLL(os.path.join(dll_dir, dll_name))
        # signatures
        self.lib.AHK_New.restype = ctypes.c_void_p
        self.lib.AHK_Connect.argtypes = [ctypes.c_void_p]; self.lib.AHK_Connect.restype = ctypes.c_bool
        self.lib.AHK_Disconnect.argtypes = [ctypes.c_void_p]
        self.lib.AHK_Sync.argtypes = [ctypes.c_void_p]; self.lib.AHK_Sync.restype = ctypes.c_bool
        self.lib.AHK_SetImageSrc_Path.argtypes = [ctypes.c_void_p, ctypes.c_char_p]
        self.lib.AHK_SetJointAngles.argtypes = [ctypes.c_void_p] + [ctypes.c_float]*6 + [ctypes.c_bool]
        self.lib.AHK_GetJointAngles.argtypes = [ctypes.c_void_p, ctypes.POINTER(ctypes.c_float)]
        self.p = self.lib.AHK_New()

    def connect(self):    return self.lib.AHK_Connect(self.p)
    def disconnect(self): self.lib.AHK_Disconnect(self.p)

    def set_eyes(self, path):          # display image
        self.lib.AHK_SetImageSrc_Path(self.p, path.encode("ascii"))

    # RAW ORDER: j1=head, j2..j5=arms, j6=waist(rotation)
    def set_pose(self, head=0., body=0., enable=True):
        self.lib.AHK_SetJointAngles(self.p, head, 0., 0., 0., 0., body, enable)

    def sync(self):          return self.lib.AHK_Sync(self.p)    # sends frame + joints

    def get_joints(self):          # needs 2 reads (C2)
        buf = (ctypes.c_float*6)()
        self.lib.AHK_GetJointAngles(self.p, buf)
        return list(buf)

```

5. tracker.py — capture + face

```

import cv2

class FaceTracker:
    def __init__(self, cam_index=0):
        self.cap = cv2.VideoCapture(cam_index, cv2.CAP_DSHOW)
        self.det = cv2.CascadeClassifier(
            cv2.data.haarcascades + "haarcascade_frontalface_default.xml")
        # v2: swap for a DNN detector (res10 SSD) for robustness

    def read_upright(self):
        ok, frame = self.cap.read()
        if not ok: return None
        return cv2.rotate(frame, cv2.ROTATE_180)    # undo 180° mounting (C4)

    def biggest_face(self, frame):
        g = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        faces = self.det.detectMultiScale(g, 1.2, 5, minSize=(60,60))
        if len(faces) == 0: return None
        x,y,w,h = max(faces, key=lambda f: f[2]*f[3])    # largest face
        return (x + w/2, y + h/2, w, h)

```

6. controller.py — control loop (error → j6)

Velocity control (incremental): the waist rotates at a rate proportional to the horizontal error until centered. Robust without needing an exact pixel→degree mapping.

```

class YawController:
    def __init__(self, cfg):
        self.cfg = cfg
        self.yaw = 0.0        # current command (degrees)
        self.err_ema = 0.0

    def update(self, face_cx, frame_w):
        if face_cx is None:    # no face -> slowly return to center
            self.yaw += (0 - self.yaw) * 0.02
            return self._clamp(self.yaw)

        err = (face_cx - frame_w/2) / (frame_w/2)    # [-1, 1]
        if abs(err) < self.cfg.deadzone: err = 0.0
        self.err_ema = (self.cfg.ema*err) + (1-self.cfg.ema)*self.err_ema

        step = self.cfg.kp * self.err_ema * self.cfg.sign    # degrees/tick
        step = max(-self.cfg.max_rate, min(self.cfg.max_rate, step))    # rate-limit
        self.yaw = self._clamp(self.yaw + step)
        return self.yaw

    def _clamp(self, v):
        lim = self.cfg.yaw_limit
        return max(-lim, min(lim, v))

```

config.py (initial values for tuning):

```

class Cfg:
    kp = 6.0; ema = 0.4; deadzone = 0.06
    max_rate = 4.0          # degrees per tick
    yaw_limit = 85.0        # margin from ±90
    sign = +1               # determine on the 1st run (which side rotates)
    eyes_path = "assets/eyes.jpg"
    dll_dir = r"dll"

```

Calibrating `sign`: on the first run, with the face to the right, check whether the waist rotates in the direction that **centers** it. If it moves away, invert `sign`.

7. `vcam.py` — output + vertical crop

```

import pyvirtualcam, cv2

class VCam:
    def __init__(self, w, h, fps=30):
        self.cam = pyvirtualcam.Camera(width=w, height=h, fps=fps)
        self.w, self.h = w, h

    def push(self, frame, face_cy=None):
        if face_cy is not None:          # re-center vertically (digital)
            shift = int(self.h/2 - face_cy)
            M = [[1,0,0],[0,1,shift]]
            frame = cv2.warpAffine(frame, __import__("numpy").float32(M),
                                   (self.w, self.h))
        self.cam.send(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
        self.cam.sleep_until_next_frame()

```

8. `main.py` — loop

```

from robot import Robot; from tracker import FaceTracker
from controller import YawController; from vcam import VCam; from config import Cfg

cfg = Cfg()
bot = Robot(cfg.dll_dir)
assert bot.connect(), "Robot failed to connect (check Zadig/driver)"
bot.set_eyes(cfg.eyes_path)

tr = FaceTracker(0)
ctl = YawController(cfg)
probe = tr.read_upright(); H, W = probe.shape[:2]
vc = VCam(W, H, 30)

try:
    while True:
        frame = tr.read_upright()
        if frame is None: continue
        face = tr.biggest_face(frame) # (cx,cy,w,h) or None
        cx = face[0] if face else None
        yaw = ctl.update(cx, W)
        bot.set_pose(head=0.0, body=yaw, enable=True)
        bot.sync() # sends eyes + joints
        vc.push(frame, face[1] if face else None)
finally:
    bot.set_pose(0,0, enable=True); bot.sync()
    bot.disconnect()

```

9. Behaviors and edge cases

Situation	Handling
No face	Waist slowly returns to center (does not get stuck in the corner)
Multiple faces	Follows the largest ($\approx$ closest). v2: lock via recognition
Oscillation	Increase <code>deadzone / ema</code> , reduce <code>kp / max_rate</code>
Brownout (servo + USB)	Move only j6 in this phase; avoid abrupt movements
Privacy	"Park" key $\rightarrow$ <code>body=0</code> , eyes closed, stop vcam
Hot-plug	Wrap <code>connect()</code> in <code>retry</code> ; detect failed <code>sync()</code> $\rightarrow$ reconnect

10. Acceptance criteria (DoD)

1. Move the face horizontally $\rightarrow$ the waist follows and **centers** it (perceived latency $< \sim 300$ ms).
2. Output appears as the "OBS Virtual Camera" device and is selectable in Zoom/Teams.
3. Digital vertical re-centering keeps the face in the central band of the output.
4. No face $\rightarrow$ robot stabilizes at center, without oscillating.
5. Clean shutdown (returns to center, disconnects).

11. Phase-specific risks

- **DLL bitness × Python** (likely x86) — use a Python of the same architecture.
- **C1**: each `sync()` sends a full frame; keep the rate at ~20–30 Hz and the eyes static.
- **C4**: 180° rotation already handled in `read_upright` ; also verify the vcam output.
- `sign` undefined until the 1st calibration.
- **OBS Virtual Camera** must be installed/registered.

12. Evolution (v2+)

- **DNN** detector (res10 SSD) or MediaPipe for robustness to angle/lighting.
- **Tracker** (KCF/CSRT) between detections for smoothness.
- Use **head pitch (j1, ±15°)** for "attention" micro-adjustments (does not move the camera, but adds life).
- Show a reaction on the display (looking toward the person) instead of static eyes.